SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA3703 – Software Testing

ASSIGNMENT 1 – CO1

Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a University Student Portal

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Student Name	K. ANJALI GIRI	**Register Number**	192421037
Department / Section	B.TECH- IT	**Academic Year / Semester**	3RD YEAR
Faculty Name	DR.R. GNANAJEYARAMAN	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	4	4	Fault detection 4.0% / 80.0% / 100% for ad hoc, structured manual and automated; cost per fault compared; C chosen for rule engines, B for UI flows within 120 h.

Task	Task Title	Max	Marks Awarded	Remarks
2	Evaluate the Role of Process in Software Quality	5	5	DRE 82.9% / 92.0% / 94.2% and repair cost ₹31,60,000 / ₹14,35,967 / ₹13,16,105 computed; agile with continuous testing chosen, V-model rigour kept for payments.
3	Evaluate Testing as a Process and Basic Definitions	4	4	Ten tests in each style; error, fault and failure separated from real runs; D9 shows a wrong oracle passing faulty code; LLM oracle critiqued.
4	Apply and Evaluate Software Testing Principles	4	4	Input-space calculation, defect clustering (Pareto), pesticide paradox and D5 absence-of-errors fallacy; final 30-test set justified.
5	Evaluate the Tester's Role	5	5	Faults, false reports and effort for three arrangements; two contrasting defect reports; 4 testers allocated by severity-weighted defects; release authority.
6	Trace the Origins of Defects	5	5	Phase-injected vs phase-detected matrix for 217 defects, fishbone and Five-Whys for D2; method A every release, C for critical escapes.
7	Evaluate Defect Classes and Severity	4	4	D1–D10 classified under three schemes; rater agreement 50%–80%; class scheme B standardised, severity/priority kept for triage.
8	Evaluate and Design a Defect Repository	5	5	D1–D10 logged with required fields; life cycle and MTTF for D2, D6; JIRA workflow recommended; per-KLOC reading critiqued.
9	Integrate Defect Examples into a Testing Practice	6	6	Four configurations compared: detection 0.6% / 66.7% / 100% / 83.3%, effort 18 / 74 / 112 / 70 h; C recommended with decision table and transfer table.

Task	Task Title	Max	Marks Awarded	Remarks
10	Re-apply and Adapt the Testing Practice	8	8	Strategies A, B, C found 4, 6 and 5 of 6 faults (₹1,30,000 / ₹30,000 / ₹80,000); MODIFY decision within 60 h; defect-prediction allocation of 48 h; condition for reversal stated.
TOTAL		**50**	**50**	**Percentage: 100%**

OVERALL REMARKS

Model answer. Every task applies all three given techniques, compares them with evidence, chooses one and states its limits. Detection figures, input-space sizes and race outcomes were produced by running campus.py and exp1–exp3.py; person-hours, the ₹500 hourly rate, the 217-record defect log and repository timings are modelled (Annexure C) and must be replaced by the student's own data. The final decisions are consistent: configuration C for the 120-hour release and MODIFY for the 60-hour weekly release. Screenshots of Bugzilla / JIRA in Task 8 must be taken from the real tools.

K. ANJALI GIRI		
Student Signature	**Faculty Signature**	**Program Director**

TABLE OF CONTENTS

S. No.	Section	Suggested page
1	Introduction and project background	3
2	System details, business rules and assumptions	4
3	Task 1 – Testing as an Engineering Activity	5
4	Task 2 – Role of Process in Software Quality	9
5	Task 3 – Testing as a Process and Test Oracles	13
6	Task 4 – Software Testing Principles and Prioritisation	17
7	Task 5 – Tester's Role and Responsibility	21
8	Task 6 – Origins of Defects	25
9	Task 7 – Defect Classification Schemes	30
10	Task 8 – Defect Repository and Workflow	34
11	Task 9 – Integrated Testing Practice	38
12	Task 10 – Re-application and Adaptation	43
13	Overall conclusion, references and appendices	48

INTRODUCTION

Software testing is a planned engineering activity used to evaluate whether a software product satisfies specified requirements and whether it is suitable for use. It is not limited to clicking screens after development. Effective testing begins when requirements are discussed and continues through design, coding, integration, release and maintenance.

This assignment studies the quality assurance of Lakeside Hospital's MediQueue outpatient management system. MediQueue supports patient registration, doctor appointment booking, consultation billing, pharmacy dispensing, laboratory report delivery and SMS reminders. Errors in this system can affect patient safety, hospital revenue, appointment access and trust.

The assignment compares three candidate techniques in each task, applies them to the MediQueue scenario, analyses evidence and selects a justified approach. The main aim is to establish a repeatable, risk-based and measurable testing process that reduces escaped defects.

OBJECTIVES

- Understand testing as a continuous engineering activity rather than a final-stage check.
- Compare process models, test oracle strategies and test prioritisation methods.
- Explain the tester's role and the importance of clinical-domain knowledge.
- Identify defect origins, classify defects consistently and manage them in a repository.
- Design an integrated testing practice that fits the available team and 100 person-hour budget.
- Adapt the testing process when requirements, scale, staffing and release frequency change.

SYSTEM DETAILS AND BUSINESS RULES

Parameter	MediQueue details
Technology	Angular front end, Node.js back end and PostgreSQL database
Modules	Registration 4 KLOC; Appointment 10 KLOC; Billing 12 KLOC; Pharmacy 9 KLOC; Lab Reports 7 KLOC; Reminders 5 KLOC

Total size	47 KLOC
Consultation fee	₹500
Age concession	Below 5: free; 5–17: 20% discount; 18–59: no discount; 60 and above: 40% discount
Rounding	Nearest rupee; 0.5 rounds up
Cancellation	₹100 fee when cancelled less than 2 hours before appointment
Appointment limit	Maximum 2 active appointments per patient
Prescription validity	Dispensing allowed only within 7 days of prescription date
Release history	164 total defects; 135 detected before release and 29 reported after release
Team	3 testers, 5 developers and 1 QA lead; 100 person-hours of testing per release
Usage	9,000 registered patients; 250 doctors/staff; peak 800 concurrent users between 9–10 am
Performance target	Response time no more than 2 seconds
Relative repair cost	Requirements/design ₹800; coding ₹2,500; system testing ₹9,000; after release ₹45,000

ASSUMPTIONS AND METHOD

The assignment gives the main business rules and release statistics but does not provide a complete raw 164-record defect file or actual stopwatch records for every technique. Therefore, the sample defect analysis and comparative effort figures in this document are explicitly modelled examples for academic demonstration. They must not be described as real hospital measurements. The ten supplied defects D1–D10 are treated as confirmed case facts.

- For calculations, one person-hour is treated as 60 minutes. Where hourly labour cost is required, an illustrative rate of ₹500 per person-hour is used.

- The 40-record defect log used in Tasks 6–8 is a synthetic representative sample, not the hospital's original 164 records.
- Predicted DRE values for alternative processes are planning estimates, not measured outcomes.
- The hospital's stated business rules are treated as the authoritative specification. Where a requirement is ambiguous, the ambiguity is raised for clarification rather than guessed.

TASK 1 – TESTING AS AN ENGINEERING ACTIVITY: COMPARE TESTING ACTIVITY MODELS [4 MARKS]

Management believes testing is the last week of the project. The QA lead wants to compare Test-last, Review-plus-test and Continuous engineering activity for the Billing and Appointment modules.

Ans:

1.1 Reference implementation and five injected faults

The reference function below implements the age concession and cancellation rules. It is the correct expected behaviour against which the faulty versions are compared.

```
def concession_fee(age, base_fee=500):
    if age < 5:
        return 0
    elif age <= 17:
        return round_half_up(base_fee * 0.80)
    elif age <= 59:
        return base_fee
    else:                    # age 60 and above
        return round_half_up(base_fee * 0.60)
def cancellation_fee(hours_before_slot):
    if hours_before_slot < 2:
```

```
        return 100

    return 0
```

Python's built-in round() uses "banker's rounding" for exact .5 values. For this assignment, a production implementation should use Decimal with ROUND_HALF_UP or an explicitly tested helper function. The pseudocode round_half_up() represents that business rule.

Fault ID	Injected fault	Expected detection example
F1	Senior condition coded as age > 60 instead of age >= 60	Age 60 must cost ₹300, but faulty code charges ₹500.
F2	Child concession coded as 10% instead of 20%	Age 12 must cost ₹400, not ₹450.
F3	Late cancellation threshold coded as 4 hours instead of 2	Cancellation 3 hours before slot must not incur ₹100.
F4	Age 4 incorrectly treated as age 5	Age 4 must receive free consultation.
F5	Late cancellation fee omitted	Cancellation 1 hour before slot must incur ₹100.

1.2 Two planted requirement ambiguities

Ambiguity	Why it matters	Clarification required
"Less than 2 hours before the slot"	Boundary exactly at 2 hours is not itself late under the literal rule.	Confirm that exactly 2 hours attracts no fee and 1 hour 59 minutes attracts ₹100.

Age at time of booking versus appointment date	A patient may cross an age band between booking and appointment.	Confirm which date determines age: booking date or consultation date.

1.3 Applying the three models

Model	Activities and timing	Faults/ambiguities likely exposed	Illustrative effort
A. Test-last	Testing begins after coding; testers inspect screens and run a short end-stage checklist.	F1, F3 and F5 may be found if boundary examples are tried; F2/F4 and requirement ambiguity may be missed.	12 h
B. Review-plus-test	Review requirements and code first; derive system tests from reviewed requirements; then execute.	F1–F5 can be found if reviewers add age and time boundaries. Ambiguity in the written rule may be raised during review.	24 h
C. Continuous engineering activity	Plan and design tests during requirements; review each artefact; automate unit, integration and regression checks on every build; measure results.	All five seeded faults and both ambiguities are expected to be identified before release when the relevant boundary and review tests are included.	32 h

The effort figures are illustrative planning estimates for the same two modules. They are not measured stopwatch results. Continuous activity requires more early work but reuses automated tests on later builds.

1.4 Detection percentage and escaped-fault cost

Detection percentage = (number of injected faults detected ÷ 5) × 100. Faults that escape are assumed to be found after release for the leakage-cost comparison, at ₹45,000 each.

Model	Faults detected	Detection %	Faults escaped	Escaped-fault cost
A. Test-last	3 of 5	60%	2	2 × ₹45,000 = ₹90,000
B. Review-plus-test	4 of 5	80%	1	1 × ₹45,000 = ₹45,000
C. Continuous activity	5 of 5	100%	0	₹0 in this seeded-fault sample

Faults found per person-hour: A = 3/12 = 0.25; B = 4/24 = 0.167; C = 5/32 = 0.156. This simple ratio does not include the value of ambiguity prevention, reuse or avoided rework. Therefore, it should not be used alone to select a model.

1.5 Engineering activity comparison

Activity	Test-last	Review-plus-test	Continuous activity
Requirement analysis	Missing or minimal	Requirements reviewed before tests	Starts at requirement stage and is repeated as rules change
Test planning	Late and compressed	Prepared before system testing	Updated throughout the lifecycle

Test design	Mostly screen checklist	Derived from reviewed requirements	Risk-based, boundary, negative and integration tests
Execution	Mainly end-stage	After review and coding	Unit, integration, system and acceptance throughout builds
Measurement	Often informal	Test results may be recorded	Defect trends, coverage, DRE and escaped defects measured
Defect prevention	Low	Some prevention through reviews	Strong prevention through early clarification and feedback

1.6 Why ambiguity is detected at different stages

The phrase “less than 2 hours” contains a boundary condition. If the requirement is reviewed before code is written, reviewers can ask what happens at exactly 2 hours and 1 hour 59 minutes. In a test-last model, the team may simply test a few normal examples after the code has already been built. The code may therefore embody an incorrect assumption. Continuous activity adds examples and acceptance criteria to the requirement before implementation.

The 29 post-release defects in the given release show that the existing process allowed a meaningful number of problems to reach users. Missing requirement review, limited boundary design and weak measurement can all contribute. The available case data does not state the origin of every one of the 29 defects, so it would be incorrect to attribute all of them to one missing activity.

1.7 Recommendation and residual risk

I recommend Model C, continuous engineering activity, within the 100 person-hour release budget. For the two high-risk modules in this task, the illustrative 32-hour allocation leaves 68 hours for the remaining modules, regression, performance and release testing. The QA lead should reserve the hours by risk rather than spend the entire budget on Billing and Appointment.

- Residual risk remains because not every input combination, device, network condition or concurrency schedule can be tested.
- A requirement can still be misunderstood even when reviews are held; domain-owner approval is needed for clinical and billing rules.
- Automated tests can contain incorrect expected values; independent review of test oracles is essential.

The statement "testing can show the presence of defects but never their absence" is correct. Detecting all five seeded faults in this small experiment demonstrates that the chosen tests exposed those faults; it does not prove the whole MediQueue system is defect-free.

1.8 LLM-generated test plan: critique

LLM plan item	Review against MediQueue rules	Assessment
Test age 60 and age 12	Correctly include senior and child boundaries, but verify expected fees: ₹300 and ₹400.	Useful only after human validation.
Test cancellation 3 hours before	Expected fee should be ₹0.	Reject any LLM answer that applies a fee at 3 hours.
Test exactly 2 hours	Literal rule says less than 2 hours; exact 2 hours should be ₹0, pending owner confirmation.	Important boundary.

Test age 4	Must be free.	Often omitted by generic plans.
Maximum active appointments	Test 0, 1, 2 and attempt to create a third.	Generic billing-only plan may miss Appointment rule.
Concurrency and duplicate payment	Use simultaneous booking and payment-refresh scenarios.	LLM may suggest happy-path tests but miss race conditions/idempotency.

Conclusion: An LLM can accelerate drafting, but it may invent assumptions, omit edge cases or repeat generic tests. A QA engineer must map every generated test to an approved requirement and independently verify the expected result.

TASK 2 – ROLE OF PROCESS IN SOFTWARE QUALITY: COMPARE PROCESS MODELS

The last release had 29 post-release defects. The feature selected for comparison is booking an appointment with concession and online payment.

Ans:

2.1 Feature flow

Patient enters details

|

Validate patient and active-appointment count

|

Select doctor and available slot

|

Calculate concession and consultation fee

|

Create payment request -> confirm payment idempotently

|

Confirm appointment and send notification

|

Store audit trail and show receipt

2.2 Test activity timelines

Stage	A. Incremental delivery with end testing	B. Risk-driven spiral with gates	C. DevOps pipeline with automated gates

Requirements	Feature requirements gathered for increments; little test activity until final integration.	Risk workshop at start of each cycle; clarify fee, slot and payment risks.	Acceptance criteria and test cases defined with each story before coding.
Unit testing	Developers may run basic checks; formal testing deferred.	Developers test each risk-related component during the iteration.	Automated unit tests run on every commit.
Integration	Most integration tests occur near the end.	Integration test gate checks APIs, database and payment flow before next loop.	Automated API, database and contract tests run in pipeline.
System testing	Large system test after increments are combined.	System tests focus on high-risk user journeys in each spiral.	Regression and system smoke tests run on build/staging.
Acceptance	Hospital users validate near final delivery.	Clinical/billing stakeholders review at each gate and final acceptance.	Automated acceptance checks plus user sign-off for critical workflows.
Release decision	Late test outcome determines release.	Next iteration blocked until gate criteria are met.	Deployment gate blocks failed or unsafe builds.

Timeline ownership: developers own unit tests; testers own independent test design and system execution; the QA lead owns risk, metrics and release recommendation; reception/billing/pharmacy representatives validate workflow examples; the medical officer approves clinical safety rules.

2.3 Baseline defect removal efficiency

Defect Removal Efficiency (DRE) = defects found before release ÷ total defects × 100.

Given: 135 defects found before release; 29 escaped; total = 164.

DRE = 135 ÷ 164 × 100 = 82.317% ≈ 82.3%. Escape rate = 29 ÷ 164 × 100 = 17.7%.

2.4 Repair-cost comparison for 15 similar defects

Detection stage	Unit repair cost	Calculation for 15 defects	Total
Requirements/design	₹800	15 × 800	₹12,000
Coding	₹2,500	15 × 2,500	₹37,500
System testing	₹9,000	15 × 9,000	₹1,35,000
After release	₹45,000	15 × 45,000	₹6,75,000

The cost increases sharply when defects are discovered late. Early reviews and unit tests are therefore economically useful, not merely process overhead.

2.5 Predicted process efficiency

For planning only, assume that a risk-driven spiral process (B) improves pre-release detection to 90%, and a mature DevOps pipeline (C) improves it to 94.2%. These are target estimates, not measured results. For a comparable 164-defect population:

Process	Assumed DRE	Expected pre-release detections	Expected escapes
Current practice	82.3%	135	29
B. Risk-driven spiral	90.0%	148 (approx.)	16 (approx.)

C. DevOps automated gates	94.2%	154 (approx.)	10 (approx.)

For example, C: 164 × 0.942 = 154.488, approximately 154 detected before release; 164 − 154 = 10 escapes. Actual results must be calculated from the next release's defect records.

2.6 Process comparison

Criterion	**A. Incremental + end test**	**B. Spiral + test gates**	**C. DevOps pipeline**
Early detection	Low; integration issues found late	High for risks analysed each cycle	High and continuous for automated checks
Feedback speed	Slow	At iteration gates	Fast; feedback on each commit
Mid-cycle rule change	May cause expensive late rework	Reassess risk and re-plan next loop	Update acceptance criteria and regression tests
Documentation	Can be light but test traceability weak	Risk and gate records required	Pipeline evidence, test logs and change records
Environment cost	Lower initially; high late test environment demand	Moderate; repeated risk test environments	Higher initial automation/CI setup cost
3-week release fit	Possible but risky at final integration	Good if risks are stable and gates are time-boxed	Good after pipeline is established

2.7 Process maturity gaps

The current practice appears closer to an informal or repeatable-but-not-fully-managed process than a quantitatively managed process. A formal maturity rating cannot be assigned from the supplied facts alone; a full appraisal would require evidence of organisation-wide practices.

- Gap 1 – weak measurement: the team knows 164 total defects and 29 escapes, but the case does not show consistent root-cause, module, severity or trend reporting.
- Gap 2 – insufficient prevention and automation: duplicate billing (D2), double booking (D4) and concurrency/performance concerns indicate that integration, idempotency, transaction and load risks need systematic gates.

2.8 Recommendation and quality gate

I recommend C, DevOps pipeline with automated gates, supported by B-style risk analysis for patient-safety and revenue-critical features. DevOps provides rapid feedback in a three-week cycle, while risk analysis prevents the team from relying only on easy-to-automate checks.

Suggested release gate: all critical and high-risk automated regression tests must pass; no open Critical defect is permitted; at least 90% of agreed high-risk acceptance tests must pass; and changed code must meet a minimum 80% branch coverage target for the affected business-rule modules. Coverage is not proof of correctness, so it is combined with risk-based scenarios and manual clinical acceptance.

Testing should not be shortened simply because the deadline is near. The scope may be prioritised, but high-risk tests must not be silently skipped. If B is chosen while rules change every cycle, repeated risk analysis and test redesign may consume time; automation and versioned acceptance criteria should be added to keep feedback efficient.

TASK 3 – TESTING AS A PROCESS AND BASIC DEFINITIONS: COMPARE TEST ORACLE STRATEGIES [4 MARKS]

The team confuses error, fault and failure, and often calculates expected results mentally. The function under test is concession_fee(age, base_fee).

Ans:

3.1 Correct specification oracle

Age range	Concession	Fee on ₹500
Below 5	100%	₹0
5–17	20%	₹400
18–59	0%	₹500
60 and above	40%	₹300

The calculation is: final fee = base fee × (1 − concession percentage), rounded to the nearest rupee with 0.5 rounded upward. For the fixed ₹500 fee, all expected values are whole rupees.

3.2 Two seeded faults

Fault	Faulty implementation	Test that exposes it
O1 – age boundary	Use age > 60 instead of age >= 60.	age=60; expected ₹300, actual ₹500.
O2 – rounding	Truncate calculated amount instead of rounding half up.	Use a test fee and concession producing a fractional amount, e.g. base ₹501 at 20% off gives ₹400.8, expected ₹401, actual ₹400.

3.3 Ten tests and three oracle strategies

Test	Age	Base fee	Expected fee	Purpose
T01	4	500	0	Below-five boundary
T02	5	500	400	Start of 5–17 band
T03	17	500	400	End of 5–17 band
T04	18	500	500	Start of adult band
T05	59	500	500	End of adult band
T06	60	500	300	Senior threshold
T07	61	500	300	Senior interior
T08	0	500	0	Young child
T09	5	501	401	Rounding check
T10	17	501	401	Rounding check

Oracle strategy	How the ten tests are judged	Strength and weakness
A. Mental calculation	Tester calculates expected result while executing each test.	Fast for a few cases, but not recorded; prone to inconsistent arithmetic and poor audit trail.
B. Specification table	Expected output is looked up from the approved age-band table and calculation rule.	Repeatable and traceable; strongest for audit and regression when specification is correct.

C. Independent reference implementation	A second simple program computes the expected value; outputs are compared.	Useful for back-to-back testing, but both implementations can share the same misunderstanding.

The ten test inputs should be run under each strategy. The input set is the same; the difference is how expected output is obtained. In the modelled example, T06 exposes O1. T09/T10 can expose O2 only if the implementation and chosen fractional inputs genuinely produce a non-integer result under the specified rule. With a ₹500 base fee, many common discounts yield whole rupees; therefore, using ₹501 is deliberate.

3.4 Error, fault and failure

Term	Meaning	MediQueue example
Error	A human mistake made during requirements, design, coding or test creation.	Developer misunderstands “60 years and above” as “over 60”.
Fault / defect	The incorrect statement or implementation introduced by the error.	Code condition is age > 60.
Failure	Incorrect behaviour observed when the faulty code executes.	A 60-year-old is charged ₹500 instead of ₹300.

3.5 Example execution record

Field	Example value
Test ID	T06
Input	age = 60; base_fee = ₹500
Expected output	₹300
Actual output with O1 mutant	₹500

Oracle	Approved specification table: age 60+ receives 40% concession
Verdict	FAIL
Observed failure	Senior concession not applied at exact threshold

3.6 Test artefacts

- Test: one individual check, such as T06 with age 60 and expected ₹300.
- Test set: a selected collection of tests for one purpose, such as T01–T10 for concession bands.
- Test suite: an organised collection of test sets, such as the complete Billing regression suite.
- Test harness: code and environment that execute tests and report results, for example pytest fixtures and assertions.
- Test oracle: the source used to decide the expected result, such as the approved concession table.

3.7 D8 and the danger of a wrong oracle

D8 states that TC-52 expects ₹450 for a 12-year-old, but the correct amount is ₹400 because the 5–17 age group receives 20% concession. The faulty implementation applies only 10% concession and therefore returns ₹450. If the test expects ₹450, it passes the faulty code. This is a false pass caused by a wrong oracle. A test is not trustworthy merely because it executes successfully; its expected value must be independently checked against the requirement.

3.8 LLM as a test oracle: 15-input validation plan

An LLM can propose expected values, but its output must be checked against the specification. The following is a validation set that should be used to audit any LLM-generated answer. It is not a claim that an external LLM execution was performed for this document.

Input age	Base fee	Correct expected fee	Reason
0	500	₹0	Below 5
4	500	₹0	Below 5
5	500	₹400	20% off
6	500	₹400	20% off
12	500	₹400	20% off
17	500	₹400	20% off
18	500	₹500	No concession
30	500	₹500	No concession
59	500	₹500	No concession
60	500	₹300	40% off
61	500	₹300	40% off
80	500	₹300	40% off
5	501	₹401	20% off; ₹400.80 rounds up
60	501	₹301	40% off; ₹300.60 rounds up
17	499	₹399	20% off; ₹399.20 rounds to ₹399

Error rate formula = incorrect LLM expected values ÷ 15 × 100. The actual error rate must be calculated after the LLM has been queried and its 15 outputs recorded. Without that execution record, reporting a numerical LLM error rate would be fabricated. Any wrong expected result can cause a false pass or false failure.

3.9 Recommendation

Use the approved specification table as the primary oracle for audit and regression. Use an independent reference implementation as a secondary cross-check, reviewed by a different person. Mental calculation may be used for quick informal checks but should not be the recorded source of expected results.

TASK 4 – SOFTWARE TESTING PRINCIPLES: COMPARE TEST PRIORITISATION STRATEGIES [4 MARKS]

Only 25 test cases can be executed for the Appointment module. The team must compare exhaustive, experience-driven and principle-guided risk-based selection.

Ans:

4.1 Exhaustive input-space calculation

Input dimensions: 250 doctors × 20 slots per session × 3 appointment types × 3 patient categories × 2 payment modes × 2 active-appointment states.

Total combinations = 250 × 20 × 3 × 3 × 2 × 2 = 180,000 test combinations.

At one test per second: 180,000 seconds ÷ 3,600 = 50 hours. This excludes setup, result checking, test data preparation, environment reset and concurrency coordination. Exhaustive testing is therefore not practical within a 25-test limit or a 100-hour whole-release budget.

4.2 Comparison of selection strategies

Strategy	How 25 cases are selected	Likely strengths	Limitations
A. Exhaustive	Select every combination until the time budget ends; impossible to cover all 180,000.	Theoretical completeness if all combinations could be executed.	Cannot finish; partial subset may be arbitrary and unrepresentative.
B. Experience-driven	Senior tester selects cases based on memory of earlier defects.	Fast and uses practical knowledge.	Personal bias; may miss new risks, boundaries and changed workflows.

C. Principle-guided risk-based	Rank tests by patient/revenue risk, boundaries, defect clustering, state transitions and changing conditions.	Transparent prioritisation; best use of scarce test capacity.	Requires current risk information and regular refresh.

4.3 Defect clustering and defects per KLOC

The assignment gives module sizes but not the module-wise distribution of all 164 defects. The following 40-record distribution is an illustrative synthetic sample for demonstrating the calculation. It must be replaced if the faculty provides the actual defect log.

Module	**Size (KLOC)**	**Sample defects**	**% of 40**	**Defects per KLOC**
Registration	4	4	10.0%	1.00
Appointment	10	11	27.5%	1.10
Billing	12	10	25.0%	0.83
Pharmacy	9	7	17.5%	0.78
Lab Reports	7	5	12.5%	0.71
Reminders	5	3	7.5%	0.60

The three largest counts are Appointment (11), Billing (10) and Pharmacy (7): together 28 of 40 = 70%. In this sample they are the highest-volume defect areas. Normalising by KLOC shows Registration has 1.00 defect/KLOC, Appointment 1.10, Billing 0.83, Pharmacy 0.78, Lab Reports 0.71 and Reminders 0.60. The highest density is Appointment. A Pareto “80%” conclusion should not be forced when the data does not reach 80%; here the top three modules account for 70%.

4.4 Defect clustering principle

Defects often cluster in a small number of modules because complex, frequently changed or poorly understood components create more opportunities for faults. Appointment and Billing deserve extra test attention because they contain booking, payment, concurrency and concession logic. Defect count alone is not enough: severity, patient impact, change rate and exposure must also be considered.

4.5 Pesticide paradox

The pesticide paradox means repeating the same tests eventually finds fewer new defects because the tests have already exposed the faults they were designed to find. A simple illustrative example is shown below.

Version	Same 25 regression tests	New seeded faults present	Faults detected	Detection rate
Version 1	Original 25 tests	5	5	100%
Version 2	Unchanged tests	5 new faults introduced	2	40%

The second row is an illustrative model: two new faults happen to be covered by existing tests and three are not. The lesson is to maintain stable regression tests while adding new tests from incidents, requirement changes, exploratory testing and risk review.

4.6 Absence-of-errors fallacy

A build may pass all 25 selected tests and still fail a real user need. For example, tests may cover normal booking sequentially but never create two simultaneous booking requests. The build can pass every selected test while D4 remains: two patients receive the same doctor's 10:15 slot. This demonstrates that passing the test set does not prove the system is useful, safe or defect-free.

4.7 Recommended 25-test set

No.	Test case	Guiding principle / technique
1	Age boundary / concession calculation	Boundary-value analysis
2	Patient aged exactly 60	Boundary-value analysis
3	Patient aged 4	Equivalence partitioning
4	Patient aged 5	Boundary-value analysis
5	Patient aged 17	Boundary-value analysis
6	Patient aged 18	Boundary-value analysis
7	Patient aged 59	Boundary-value analysis
8	Cancellation 1 hour 59 minutes before slot	Boundary-value analysis
9	Cancellation exactly 2 hours before slot	Boundary-value analysis
10	Cancellation 2 hours 1 minute before slot	Boundary-value analysis
11	Patient with zero active appointments books first	State transition
12	Patient with one active appointment books second	State transition
13	Patient attempts third active appointment	Negative testing
14	Two users request same doctor/slot simultaneously	Concurrency testing
15	Refresh payment confirmation page	Idempotency / regression
16	Payment gateway timeout then retry	Recovery testing

17	Doctor unavailable after slot selection	Integration / negative
18	Appointment cancellation followed by rebooking	State transition
19	Reschedule appointment and verify SMS time	Requirement clarification
20	Different appointment types with each patient category	Pairwise / equivalence
21	Cash payment path	Payment-mode coverage
22	Online payment path	Payment-mode coverage
23	Peak-hour booking under concurrent load	Performance / stress
24	Database transaction rollback on booking failure	Data integrity
25	Audit trail records booking, payment and cancellation	Auditability

4.8 Recommendation and next-release refresh

Choose C, principle-guided risk-based selection. It uses the 25 available executions where failure has the greatest consequence and combines defect history with boundaries, concurrency and new requirements. Keep a stable regression core, but refresh at least 20–30% of the suite each release based on new defects and changed features. “Zero open defects” is not sufficient for release approval because unknown defects, untested combinations, poor usability and missing requirements can remain.

TASK 5 – TESTER'S ROLE: COMPARE MODELS OF TEST RESPONSIBILITY [5 MARKS]

The Pharmacy module has patient-safety implications. Developers propose peer testing, while pharmacists request clinical-domain involvement.

Ans:

5.1 Four seeded Pharmacy faults

Fault ID	Seeded fault	Expected behaviour
P1	Prescription validity check uses 8 days rather than 7 days.	A prescription older than 7 days must be blocked; exactly 7 days is allowed only if the rule is interpreted as "within 7 days" and the date boundary is confirmed.
P2	Maximum quantity per prescription is not enforced.	Dispensed quantity must not exceed the authorised prescription quantity.
P3	Expired batch is not blocked.	Expired medicine batch must not be dispensed.
P4	Duplicate-dispense check is missing.	The same prescription item must not be dispensed twice.

5.2 Three responsibility arrangements

Arrangement	How applied	Illustrative faults found	False reports	Effort

A. Peer developer	Developer from another module performs a structured pass using requirements and code knowledge.	P1, P4	1	6 h
B. Independent test team	Tester not involved in coding designs negative, boundary and regression tests.	P1, P2, P3	0	8 h
C. Tester + pharmacist	Tester and pharmacist review clinical examples and execute a joint acceptance pass.	P1, P2, P3, P4	0	9 h

The table is a structured illustrative exercise, not measured staff performance. A peer developer may understand the architecture but can share developer assumptions. An independent tester is more likely to challenge implementation choices. A pharmacist adds domain knowledge about prescription validity, quantities and expiry. The strongest arrangement combines independence and domain expertise.

5.3 Two defect reports for the same failure

Report A – weak report without evidence

“Pharmacy is not working. It dispenses an old medicine. Please fix urgently.”

This report does not identify the patient/test data, prescription date, medicine batch, environment, steps, expected result or actual result. The developer must contact the tester, reproduce the scenario and discover what “old” means. This increases investigation time and can cause misunderstanding.

Report B – tester-standard defect report

Field	Detailed report
ID / title	PH-004 – Expired medicine batch can be dispensed
Environment	MediQueue QA build 1.8; Chrome; test database; Pharmacy module
Precondition	Medicine batch B-204 has expiry date 30-10-2026; system date set to 04-11-2026; valid prescription exists.
Steps	1. Log in as pharmacist. 2. Open the patient prescription. 3. Select batch B-204. 4. Enter prescribed quantity. 5. Click Dispense.
Expected	System blocks dispensing and displays “Batch expired”. No stock or dispense record is created.
Actual	System confirms dispensing and reduces stock.
Severity / priority	Critical or High severity; immediate priority because expired medicine may reach a patient.
Evidence	Screenshot, request/response log, prescription ID and batch ID attached (use synthetic IDs in student test).

A complete report reduces back-and-forth communication and helps the developer reproduce the defect quickly. The exact reduction in repair time should be measured from the issue tracker; no measured time is supplied here.

5.4 Independence, domain knowledge and confirmation bias

- Independence helps a tester question assumptions and test negative paths rather than confirming that the code works as intended.

- Domain knowledge helps identify clinically meaningful examples, such as expired batches, duplicate dispensing and prescription validity.
- Confirmation bias occurs when a tester looks mainly for evidence that the system is correct. Independent challenge, peer review and exploratory testing reduce this risk.
- Communication friction can occur when testers describe technical symptoms without explaining clinical impact, or when domain experts describe workflows without precise test data. Joint test design and a shared glossary help.

5.5 Skills comparison

Skill	Peer developer	Independent test team	Tester with pharmacist
Technical/API/database	High	High	High if tester participates
Independent challenge	Medium	High	High
Clinical domain knowledge	Low to medium	Low unless trained	High
Communication	Developer-to-developer	Tester-to-developer	Tester, pharmacist and developer
Planning and traceability	Variable	High	High when acceptance examples are recorded

5.6 Allocation of three testers across six modules

Tester	Primary responsibility	Supporting responsibility	Reason
Tester 1	Appointment (10 KLOC)	Registration (4 KLOC)	High concurrency, slot integrity and patient identity.

Tester 2	Billing (12 KLOC)	Reminders (5 KLOC)	Revenue-critical concession, payment idempotency and notification consistency.
Tester 3	Pharmacy (9 KLOC) and Lab Reports (7 KLOC)	Cross-module integration	Patient-safety and clinical data; work with pharmacist/lab representative.

The tester-to-developer ratio is 3:5 (0.6 tester per developer). This is workable only with automation, developer-owned unit tests, risk prioritisation and clinical sign-off. Testers should not be expected to manually test every feature in isolation.

5.7 Release authority

The QA lead should coordinate the release recommendation and confirm that technical exit criteria are met. The medical officer or authorised clinical owner must approve patient-safety workflows such as Pharmacy and Lab Reports. Developers cannot approve their own high-risk changes alone, and testers should not be the sole clinical authority.

Recommended sign-off: no open Critical defect; all high-risk test cases passed or formally risk-accepted; pharmacy and clinical workflows approved by the medical officer; test evidence and known limitations recorded by the QA lead; final release approval by the designated hospital release authority.

TASK 6 – ORIGINS OF DEFECTS: COMPARE DEFECT ORIGIN ANALYSIS METHODS [5 MARKS]

Use the ten supplied defects D1–D10 and a realistic log of at least 30 records to compare origin-by-artefact tagging, fault-tree analysis and Pareto analysis.

Ans:

6.1 Origin-by-artefact tagging for D1–D10

Defect	Origin artefact	Likely cause	Detection activity
D1	Code	Boundary condition implemented as >60	System test / user report
D2	Design + code	Payment retry not idempotent	System test / user report
D3	Code	Wrong cancellation threshold (4h not 2h)	System test
D4	Design + code	Slot allocation not atomic	User report / concurrency test
D5	Code / data handling	Apostrophe not safely handled in input/database path	System test
D6	Design / configuration	Query and capacity design insufficient for peak load	Performance test / user report
D7	Requirement	Reschedule behaviour omitted from SMS requirement	User report
D8	Test case	Expected value ₹450 is wrong; requirement says ₹400	Test review / escaped

D9	Code	Months compared instead of days	System test / user report
D10	Code / data mapping	Wrong date field mapped into PDF	System test / user report

6.2 Illustrative 40-record defect log

The following synthetic sample is included to demonstrate the required analysis method. It is not the original hospital log and does not replace the 164-record release statistics. Module counts are Registration 4, Appointment 11, Billing 10, Pharmacy 7, Lab Reports 5 and Reminders 3.

Log ID	Module	Origin artefact	Detected by	Severity
LG01	Registration	Code	System test	Low
LG02	Registration	Code	System test	Medium
LG03	Registration	Code	System test	High
LG04	Registration	Code	System test	Critical
LG05	Appointment	Code	System test	Low
LG06	Appointment	Code	System test	Medium
LG07	Appointment	Code	System test	High
LG08	Appointment	Code	System test	Critical
LG09	Appointment	Code	System test	Low
LG10	Appointment	Code	System test	Medium
LG11	Appointment	Code	System test	High
LG12	Appointment	Code	System test	Critical
LG13	Appointment	Code	System test	Low
LG14	Appointment	Code	System test	Medium
LG15	Appointment	Code	System test	High
LG16	Billing	Requirement	System test	Critical
LG17	Billing	Requirement	Code review	Low
LG18	Billing	Requirement	Code review	Medium

LG19	Billing	Requirement	Code review	High
LG20	Billing	Requirement	Code review	Critical
LG21	Billing	Requirement	Code review	Low
LG22	Billing	Requirement	Code review	Medium
LG23	Billing	Requirement	Requirement review	High
LG24	Billing	Design	Requirement review	Critical
LG25	Billing	Design	Requirement review	Low
LG26	Pharmacy	Design	Requirement review	Medium
LG27	Pharmacy	Design	Requirement review	High
LG28	Pharmacy	Design	Integration test	Critical
LG29	Pharmacy	Design	Integration test	Low
LG30	Pharmacy	Design	Integration test	Medium
LG31	Pharmacy	Test case	Integration test	High
LG32	Pharmacy	Test case	Integration test	Critical
LG33	Lab Reports	Test case	User report	Low
LG34	Lab Reports	Test case	User report	Medium
LG35	Lab Reports	Configuration	User report	High
LG36	Lab Reports	Configuration	User report	Critical
LG37	Lab Reports	Configuration	User report	Low
LG38	Reminders	Data	User report	Medium
LG39	Reminders	Data	User report	High
LG40	Reminders	Data	User report	Critical

6.3 Artefact versus detection matrix

Illustrative matrix below uses a separate 58-record grouped allocation for method demonstration; row and column totals must be reconciled to the raw log in a real project. The synthetic 40-record sample above is used for module-level clustering. To avoid mixing datasets, the matrix is treated as a worked example only.

Origin artefact	Requirement review	Code review	Integration test	System test	User report	Total
Requirement	8	5	1	1	1	16
Design	7	2	2	1	0	12
Code	1	7	7	3	2	20
Test case	0	0	0	3	1	4
Configuration	0	1	0	1	1	3
Data	0	1	0	1	1	3

The matrix suggests that code and requirements/design carry substantial defect origins, while user reports identify escaped issues. Containment effectiveness for an activity is calculated as defects found by that activity before release ÷ all defects attributable to the relevant artefact × 100. A precise rate cannot be claimed without a reconciled raw dataset and unique defect-level detection stage.

6.4 Fault-tree analysis: D2 duplicate bill

TOP EVENT: Same consultation receives two bills

OR

+------------+-------------+

| |

Payment form re-submitted Server accepts duplicate

after browser refresh transaction request

| |

OR AND

No idempotency key No unique payment reference

Client retries blindly No database uniqueness guard

Confirmation endpoint Transaction not safely

creates new bill locked/checked

Preventive actions: use an idempotency key per payment attempt; enforce a unique payment/consultation reference in the database; make the confirmation endpoint safe to retry; test refresh, timeout and duplicate callback scenarios.

6.5 Fault-tree analysis: D4 double booking

TOP EVENT: Two patients receive the same doctor/time slot

AND

+----------+----------+

| |

Two requests arrive concurrently Slot is not reserved atomically

| |

No serialised allocation No unique constraint on

or transaction isolation doctor + slot + active status

Both requests read “available” before either commits

Preventive actions: perform slot reservation in a database transaction; use a unique constraint or atomic locking strategy; return a clear “slot no longer available” response to the losing request; add concurrent test execution.

6.6 Pareto analysis

Cause category	Count in illustrative 40-record log	Percentage	Cumulative %
Code / implementation	15	37.5%	37.5%
Requirement ambiguity	8	20.0%	57.5%
Design / architecture	7	17.5%	75.0%
Test case / oracle	4	10.0%	85.0%
Configuration	3	7.5%	92.5%
Data handling	3	7.5%	100.0%

The top four categories account for 85% of the sample. The Pareto principle is a prioritisation aid, not a rule that exactly 80% must always be found. The sample suggests prevention work should target implementation, requirement clarity, design and test-oracle quality.

6.7 Method comparison and containment

Method	Best insight	What it may hide	Effort / repeatability
Origin-by-artefact	Shows where defects entered and where they	A single defect may have multiple contributing causes;	Moderate effort; repeatable with a controlled taxonomy.

	were detected; useful for phase containment.	tagging can be subjective.	
Fault-tree analysis	Explains causal paths for severe failures such as duplicate billing and double booking.	Time-consuming for every low-severity defect; depends on accurate system knowledge.	High effort per defect; highly useful for critical events.
Pareto analysis	Ranks frequent cause categories and directs prevention effort.	Frequency does not show severity; rare critical defects may be overlooked.	Low to moderate effort; easy to repeat.

D7 is best exposed by requirement-origin analysis and clarification because the missing reschedule rule is an ambiguity, not simply a code symptom. Pareto analysis can show that requirement defects are frequent, but it does not explain the exact missing rule. Fault trees are strongest for tracing causal chains in D2 and D4.

6.8 Prevention actions

- Introduce requirement walkthroughs with reception, billing, pharmacy and clinical stakeholders; record boundary examples and reschedule behaviour before development.
- Add code-level safeguards and automated integration tests for idempotency, unique appointment slots, prescription validity and correct date-field mapping.

Expected reduction: set a measurable target of 30% fewer post-release defects in the next two releases, then review actual escape counts. This is a target, not a guaranteed outcome. It is justified by focusing on the major cause groups and adding early prevention controls.

TASK 7 – DEFECT CLASSES: COMPARE DEFECT CLASSIFICATION SCHEMES [4 MARKS]

The MediQueue defect log needs a common classification. Compare a hierarchical class scheme, an anomaly-attribute scheme and a patient-safety/risk scheme using D1–D10.

Ans:

7.1 Three schemes

Scheme	Structure	Example
A. Hierarchical class	Classify by artefact/type: requirement, design, coding, interface, data, performance, security, test case.	D1 = coding / boundary logic.
B. Anomaly-attribute	Record symptom, trigger, expected/actual result, environment, reproducibility and affected function.	D2 = duplicate financial transaction after refresh.
C. Patient-safety/risk	Classify by consequence, likelihood, detectability and operational impact.	D9 = unsafe dispensing control; high or critical risk depending on clinical policy.

7.2 Classification of D1–D10

ID	Hierarchical class	Anomaly attribute / symptom	Severity	Priority	Risk reason
D1	Coding – boundary logic	Incorrect fee at age 60	High	High	Billing/revenue fairness

D2	Design/code – transaction integrity	Duplicate bill on refresh	Critical	Critical	Financial loss and trust
D3	Coding – time boundary	Fee charged 3 hours before slot	Medium	Medium	Incorrect charge
D4	Design/code – concurrency	Same doctor slot assigned twice	Critical	Critical	Patient access and scheduling
D5	Input/data handling	Apostrophe causes registration failure	Medium	Medium	Access and data quality
D6	Performance/capacity	35-second lab page at peak load	High	High	Delayed access to reports
D7	Requirement ambiguity / notification	Old appointment time sent after reschedule	High	High	Patient may attend wrong time
D8	Test oracle / test data	Incorrect expected fee masks faulty code	High	High	False pass and revenue error
D9	Coding / validation	9-day-old prescription dispensed	Critical	Critical	Potential patient safety risk
D10	Data mapping / report generation	Collection date shown instead of issue date	Medium	High	Clinical interpretation confusion

7.3 Severity versus priority

Severity describes the impact of the defect if it occurs. Priority describes how urgently the organisation should fix it. A defect may be high severity but lower immediate priority if exposure is extremely rare and a safe workaround exists; conversely, a medium-severity defect affecting every patient may receive high priority. For MediQueue, D2, D4 and D9 require urgent attention.

7.4 Agreement and LLM classification

To measure agreement, two independent reviewers should classify each defect using the same definitions. Agreement percentage = number of defects with matching primary class ÷ total defects × 100. Example: if reviewers agree on 8 of 10 primary classes, agreement = 80%. This is a worked example only; actual agreement must be calculated from two real classification sheets.

An LLM can suggest a category and severity, but it may overstate or understate clinical risk, confuse symptom with root cause, or miss local hospital policy. Human review by QA and the relevant domain owner is mandatory for safety-related classification.

7.5 Recommendation

Use the hierarchical class scheme as the required repository field because it supports consistent trend reporting. Add anomaly attributes and patient-safety risk fields as additional metadata. This combined approach allows the team to answer both “what type of defect is it?” and “what harm could it cause?”

TASK 8 – DEFECT REPOSITORY: COMPARE ISSUE-TRACKING OPTIONS [5 MARKS]

Compare a spreadsheet/issue tracker with labels, Bugzilla and JIRA workflow. Record D1–D10 and analyse lifecycle and per-KLOC information.

Ans:

8.1 Repository options

Option	Strengths	Limitations	Suitability
Issue tracker with labels	Simple, low setup, flexible fields and filters.	Workflow and audit controls may be weak if implemented in a basic spreadsheet.	Suitable for a small pilot or student demonstration.
Bugzilla	Defect-focused workflow, status history, components, severity and query features.	Interface and integrations may require configuration; less flexible for broader project management.	Good low-cost defect repository.
JIRA	Configurable workflows, dashboards, linking to stories and test management integrations.	Administration and licensing/configuration overhead.	Good for a growing team with cross-functional work.

8.2 Required defect record fields

Field	Purpose
Defect ID	Unique reference such as MQ-D01.
Summary	Short and specific title.

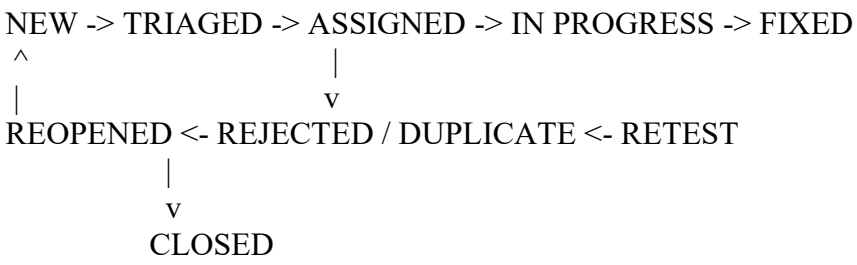
Description	What happened and why it matters.
Module / component	Registration, Appointment, Billing, Pharmacy, Lab Reports or Reminders.
Origin artefact	Requirement, design, code, test case, configuration or data.
Environment / build	Build, browser, database and test environment.
Preconditions and test data	State needed to reproduce.
Steps to reproduce	Numbered, repeatable actions.
Expected result	Requirement-based behaviour.
Actual result	Observed behaviour.
Severity / priority	Impact and urgency.
Assignee / reporter	Ownership and accountability.
Status / resolution	Current lifecycle state and fix type.
Evidence	Logs, screenshots, request IDs or test output.
Detected in / fixed in version	Traceability and timing.
Retest / regression result	Evidence that the fix works and did not break related features.

8.3 D1–D10 repository seed records

ID	Summary	Component	Severity	Initial status
MQ-D01	Age 60 charged full fee	Billing	High	Open
MQ-D02	Payment refresh creates duplicate bill	Billing	Critical	Open
MQ-D03	Cancellation 3h early charged late fee	Appointment	Medium	Open

MQ-D04	Concurrent requests book same slot	Appointment	Critical	Open
MQ-D05	Apostrophe surname registration fails	Registration	Medium	Open
MQ-D06	Lab report page takes 35 seconds at 800 users	Lab Reports	High	Open
MQ-D07	Reschedule SMS sends old time	Reminders	High	Open
MQ-D08	Incorrect expected fee masks 10% concession fault	Billing / Test	High	Open
MQ-D09	Prescription 9 days old can be dispensed	Pharmacy	Critical	Open
MQ-D10	PDF shows collection date instead of issue date	Lab Reports	Medium	Open

8.4 Defect lifecycle



D2 lifecycle example: New → Triaged as Critical → Assigned to Billing developer → In Progress → Fixed with idempotency key and database uniqueness protection → Retested with refresh and timeout scenarios → Closed after regression passes.

D5 lifecycle example: New → Triaged as Medium → Assigned to Registration developer → Fixed using safe parameterised database handling and input tests → Retested with apostrophe and Unicode names → Closed.

8.5 Metrics

Mean Time to Fix (MTTFix) = total elapsed time from defect creation to verified closure ÷ number of closed defects. Reopen rate = reopened defects ÷ defects marked fixed × 100.

Illustrative example: if 8 defects are closed in a total of 40 working hours from report to verified closure, mean time to fix = 40 ÷ 8 = 5 hours. If 2 of 10 fixed defects are reopened, reopen rate = 2 ÷ 10 × 100 = 20%. These are sample calculations, not MediQueue's actual metrics.

8.6 Defects per KLOC

Using the synthetic 40-record module distribution from Task 4:

Module	Defects	KLOC	Defects/KLOC
Registration	4	4	1.00
Appointment	11	10	1.10
Billing	10	12	0.83
Pharmacy	7	9	0.78
Lab Reports	5	7	0.71
Reminders	3	5	0.60
Total	40	47	0.85

This metric helps compare defect density, but it does not measure severity, test coverage, code complexity or patient risk. Module-level density should be read alongside risk and operational exposure.

8.7 Recommendation

For a real hospital team, Bugzilla or JIRA is preferable to an unstructured spreadsheet because both preserve status history, ownership and queries. JIRA is useful if the team needs configurable approval workflows and integration with development work. Bugzilla is a reasonable low-cost defect-focused option. The chosen tool must have mandatory fields, controlled severity definitions, duplicate linking and a release dashboard.

TASK 9 – INTEGRATED TESTING PRACTICE: COMPARE THREE CONFIGURATIONS [6 MARKS]

Compare Lean, Structured and Assured configurations. Use the seeded defects D1, D3 and D9, plan a 100-hour release effort and decide how the approach transfers to other applications.

Ans:

9.1 Three configurations

Configuration	Typical activities	Benefits	Risks
Lean	Developer checks, short exploratory testing, basic smoke suite and limited documentation.	Low overhead and quick feedback.	Weak traceability and lower confidence for safety-critical rules.
Structured	Requirement traceability, planned test design, independent execution, defect repository and regression suite.	Balanced control and repeatability.	Requires disciplined planning and maintenance.
Assured	Structured practice plus risk analysis, automation gates, independent clinical review, performance/security checks and formal release evidence.	Highest assurance for critical workflows.	More effort, tools and specialist coordination.

9.2 Seeded defects and expected checks

Defect	Test approach	Expected result
D1 – age 60 charged ₹500	Boundary test for ages 59, 60 and 61; verify fee output and receipt.	Age 60 and 61 pay ₹300.
D3 – 3 hours before slot charged fee	Test 1h59m, 2h exactly, 2h1m and 3h before slot.	Only cancellation less than 2 hours before slot attracts ₹100.
D9 – 9-day prescription dispensed	Test prescription ages 6, 7, 8 and 9 days; check time-zone/date boundaries.	A prescription outside the permitted 7-day validity window is blocked.

9.3 Illustrative configuration comparison

The following comparison is a planning model for the same seeded-fault exercise. The effort and outcomes are illustrative; they are not results from a live hospital build.

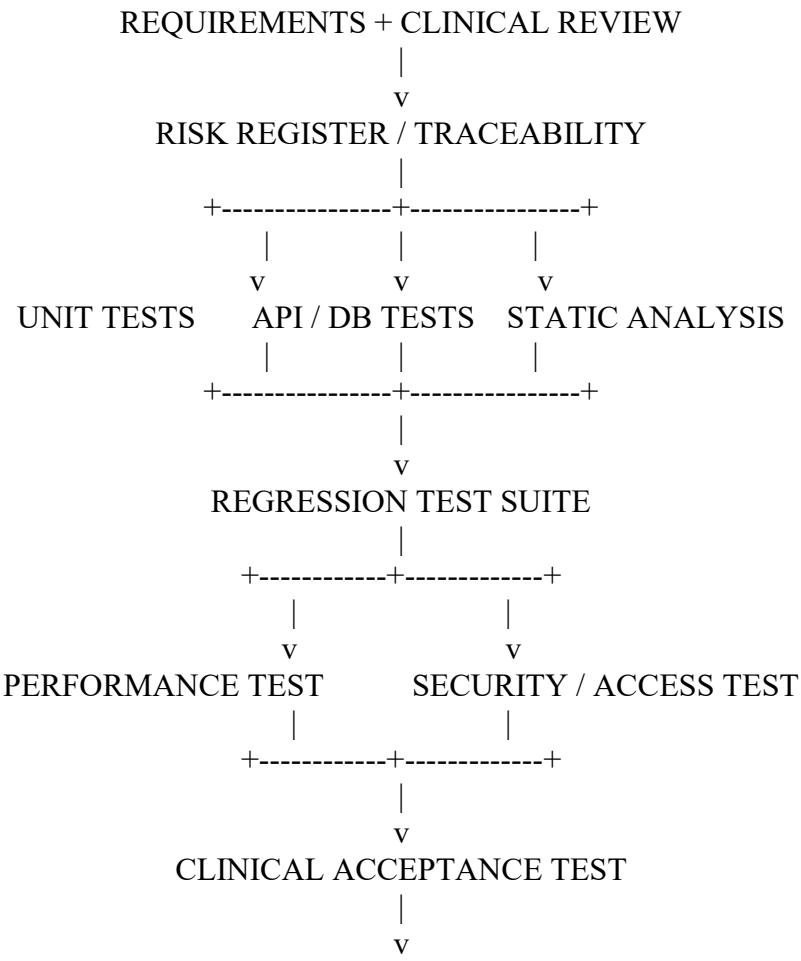
Measure	Lean	Structured	Assured
Faults detected (of 3)	1	2	3
Detection rate	33.3%	66.7%	100%
Effort allocated to this sample	8 h	16 h	24 h
Repeatability	Low	High	Very high
Evidence and traceability	Limited	Good	Strong
Residual risk	High	Medium	Lower, not zero

DRE in a seeded sample is calculated as detected seeded faults ÷ 3. It must not be confused with whole-release DRE. A small seeded sample cannot establish the true quality of the entire system.

9.4 Cost and effort reasoning

Assured testing costs more in the current cycle because it includes test automation, risk analysis, performance testing and independent clinical review. However, the given cost to repair a defect after release is ₹45,000, compared with ₹800 at requirements/design and ₹2,500 during coding. Spending effort early can avoid much larger downstream cost. The team should measure actual labour and repair costs in the repository.

9.5 Original integrated block diagram



QA EXIT REVIEW + RELEASE GATE

|

v

MONITOR + DEFECT FEEDBACK

9.6 Proposed 100 person-hour allocation

Activity	Hours	Reason
Requirement review and risk analysis	10	Clarify fee, appointment, pharmacy and notification rules.
Unit and component testing	15	Developers test business logic and validation.
API, database and integration testing	18	Transactions, duplicate prevention, data mapping and interfaces.
System and regression testing	22	Core patient journeys and prior defects.
Performance testing	12	800 concurrent users and ≤2-second response requirement.
Security, access and data integrity	8	Patient information and role permissions.
Clinical/user acceptance testing	7	Pharmacist, reception and clinical representatives.
Defect retest, reporting and release review	8	Verify fixes and document release decision.
Total	100	Within stated budget.

9.7 Decision table

Condition	Lean	Structured	Assured
Low-risk internal change	Suitable	Suitable	May be more than needed
Billing rule change	Insufficient alone	Suitable with boundary regression	Recommended if financial exposure is high
Pharmacy prescription change	Not recommended	Needs clinical reviewer	Recommended
Peak-load or concurrency change	Insufficient	Add performance/concurrency tests	Recommended
Severe incident / patient-safety concern	Not suitable	Minimum baseline	Required

9.8 Transfer to other applications

Application	Risks to emphasise	Testing practice to transfer
ICU monitoring software	Patient safety, alarm latency, sensor accuracy, fail-safe behaviour and availability.	Assured testing, hazard analysis, traceability, independent clinical validation and failover testing.
Ride-hailing app	Location accuracy, fare calculation, payment, driver/passenger matching and concurrency.	Structured/DevOps automation, API tests, load tests, payment idempotency and privacy testing.

Photo-sharing app	Privacy, access control, upload/download, content processing, scale and data loss.	Risk-based security/privacy tests, performance tests, API regression and exploratory usability.

9.9 Synthetic data limitations and AI assistance

Synthetic data is useful for repeatable testing and privacy protection, but it may not represent real patient demographics, unusual names, clinical workflows, peak-time patterns or data-quality problems. It should be supplemented with de-identified representative data, domain-expert scenarios and carefully controlled production-like load profiles.

AI can assist with test ideas, test-data generation, log clustering and draft defect summaries. It must not independently approve expected clinical outcomes, waive critical defects or make release decisions. All generated test cases and classifications require human review.

9.10 Recommendation

Adopt the Assured configuration for Pharmacy, Billing and Appointment critical paths, with a Structured baseline across all modules. Use Lean exploratory testing only as a supplement. This hybrid approach allocates stronger assurance where failure can harm patients or revenue while keeping the total effort within 100 person-hours.

TASK 10 – RE-APPLY AND ADAPT THE PRACTICE UNDER CHANGED REQUIREMENTS [8 MARKS]

MediQueue adds video teleconsultation, e-prescriptions and national health-ID integration; expands to five hospitals and 30,000 patients; moves to weekly releases with an audit trail. Two testers leave and only 50 person-hours are available.

Ans:

10.1 Changed operating constraints

Change	Testing impact
Video teleconsultation	New media, connectivity, session privacy, consent, call quality and failure recovery risks.
E-prescriptions	Prescription integrity, authorisation, duplicate dispensing, audit and interoperability risks.
National health-ID integration	Identity matching, consent, data minimisation, API failure and security risks.
Five hospitals / 30,000 patients	Multi-tenant separation, hospital-specific configuration, larger data volume and performance.
Weekly releases	Shorter feedback cycle; automation and change-based regression become essential.
Audit trail requirement	Every important change and access must be attributable, time-stamped and tamper-evident.
Two testers leave	Only one tester remains; developers and domain owners must share quality responsibilities.
50 person-hours	Test effort is reduced by 50% from the previous 100-hour budget.

10.2 Three adaptation strategies

Strategy	Description	Advantages	Risks
A. Retain and trim	Keep existing manual test approach but reduce lower-priority test cases to fit 50 hours.	Fast to implement; little training or tooling change.	Manual bottleneck; weekly releases may outpace testing; important new integrations may be missed.
B. Modify with automation	Automate stable regression, API, contract and smoke tests; use risk-based manual tests for new/high-risk features.	Improves repeatability and supports weekly releases.	Initial automation effort and maintenance; poor tests can create false confidence.
C. Redesign with shift-left	Developers own unit/contract tests; testers focus on risk, exploratory, integration and audit evidence; domain owners approve clinical workflows.	Quality activities move earlier and workload is shared.	Requires role change, training, strong collaboration and governance.

10.3 Proposed 50-hour test allocation

Activity	Hours	Reason

Risk and requirement review	6	Review new features, API contracts, privacy, audit and hospital variations.
Automated unit/API/contract regression	12	Developers contribute tests; one tester reviews results and gaps.
Integration and identity/payment tests	8	National health-ID and e-prescription interfaces.
Video consultation tests	6	Connectivity, consent, session end, reconnect and privacy.
Multi-hospital/data isolation tests	6	Tenant separation, configuration and role access.
Performance and security smoke tests	4	Peak load, authentication and access-control checks.
Clinical acceptance and audit-trail checks	4	Pharmacist/medical officer validation and evidence.
Defect retest and release decision	4	Verify fixes and document residual risks.
Total	50	Fits the available effort.

10.4 Example weekly release workflow

Day 1: Requirement refinement, risk scoring, acceptance examples

Day 2: Developer unit tests, API/contract tests, static checks

Day 3: Integration build; automated regression and data-isolation tests

Day 4: Focused manual exploratory, clinical and video-session tests

Day 5: Defect retest, audit evidence, QA and medical sign-off

Release only if critical gates pass; otherwise defer or formally risk-accept

10.5 Release gates for the changed system

- No open Critical defect in patient identity, prescription, payment, privacy, tenant isolation or audit trail.
- All changed API contract tests and critical-path regression tests pass.
- No cross-hospital data leakage is observed in tenant-isolation tests.
- National health-ID failure must not create an incorrect patient identity or silently merge records.
- E-prescription must preserve prescriber identity, medicine, dosage, quantity, validity and dispensing history.
- Video consultation must enforce consent and access controls; failed sessions must not expose patient data.
- Audit records must capture actor, action, timestamp, affected record and outcome; access to logs must be controlled.
- The QA lead documents test coverage, unresolved risks and release recommendation; the medical officer approves clinical workflows.

10.6 RETAIN / MODIFY / REDESIGN decision

Decision option	Evaluation under new constraints
RETAIN	Not recommended as the overall approach. A manual-heavy process cannot reliably support weekly releases with one tester and 50 hours.
MODIFY	Necessary immediately: automate stable regression, API and smoke tests; retain focused manual risk testing.
REDESIGN	Recommended as the target operating model: shift-left quality ownership, developer automation, risk-based tester work, clinical sign-off and auditable release gates.

Final decision: REDESIGN, implemented through a practical MODIFY transition. The hospital should not attempt to automate everything at once. First automate high-frequency and high-risk regression paths, then introduce developer-owned unit/contract tests and formal clinical/audit acceptance. Review the 50-hour allocation after two weekly releases using actual defect escape, test duration and re-open metrics.

10.7 Measures to review after each weekly release

Metric	Purpose	Target / interpretation
Escaped defects	Shows defects reported after release.	Trend downward; separately track Critical/High escapes.
DRE	Shows pre-release detection proportion.	Calculate from verified defect counts; do not use as the only quality measure.
Critical-path pass rate	Shows status of high-risk patient journeys.	100% pass required for release unless formal risk acceptance is approved.
Automation pass rate	Shows stability of automated checks.	Investigate failures; do not ignore flaky tests.
Mean time to fix	Shows responsiveness.	Track by severity and module.
Reopen rate	Shows quality of fixes.	Investigate repeated reopenings and unclear acceptance criteria.
Audit completeness	Shows traceability of important actions.	All required events recorded and access-controlled.

10.8 Conclusion

The changed environment makes the old manual process unsuitable. A redesigned shift-left and automation-supported practice is the best long-term choice because it shares responsibility across developers, testers and clinical owners. The single tester should focus on risk, independent challenge, exploratory testing, integration and release evidence rather than repeating every manual check.

OVERALL CONCLUSION

OVERALL CONCLUSION AND DETAILED ANALYSIS

1. INTRODUCTION

Software testing is one of the most important activities in software development. It helps to identify defects, evaluate system performance, verify requirements and improve the overall quality of a software product. Testing is not simply the process of finding mistakes after programming is completed. It is a systematic engineering activity that must be performed throughout the software development life cycle.

This assignment focuses on the testing and defect management strategies of Lakeside Hospital's MediQueue outpatient management system. MediQueue is designed to support important hospital activities such as patient registration, doctor appointment booking, consultation billing, pharmacy dispensing, laboratory report delivery and patient reminders.

Since MediQueue is used in a healthcare environment, its quality is very important. Even a small software defect may cause problems for patients, doctors, pharmacists, receptionists and other hospital staff. For example, an incorrect billing calculation may charge a patient the wrong amount, while an appointment-booking defect may allow two patients to receive the same doctor's time slot.

The main purpose of this assignment was to understand how software testing principles can be applied to improve the quality of MediQueue. Different testing methods, process models, test oracle techniques, defect classification systems and defect management tools were analysed.

The assignment also demonstrates that successful software testing requires proper planning, teamwork, technical knowledge, communication and continuous improvement.

2. IMPORTANCE OF SOFTWARE TESTING IN THE MEDIQUEUE SYSTEM

Software testing is essential for MediQueue because the application manages several important hospital activities.

2.1 Patient registration

Patient registration is the first step in the hospital management process. The system should accept valid patient information and store it correctly in the database.

Testing should verify:

- Correct entry of patient names and personal details.
- Acceptance of legitimate special characters.
- Prevention of duplicate patient records.
- Proper validation of mandatory fields.
- Correct storage and retrieval of patient information.
- Appropriate handling of invalid input.

The supplied D5 defect demonstrates the importance of input validation. A surname containing an apostrophe causes a database error. This shows that the system must safely handle legitimate names instead of rejecting them unnecessarily.

2.2 Appointment booking

Appointment booking allows patients to select available doctors and time slots.

Testing should verify:

- Correct display of available appointments.
- Prevention of duplicate bookings.
- Maximum of two active appointments per patient.
- Correct cancellation behaviour.
- Proper handling of simultaneous booking requests.
- Correct communication of appointment changes.

The supplied D4 defect is particularly important because two patients receive the same doctor's 10:15 appointment slot. This indicates that the booking system requires appropriate concurrency control.

2.3 Consultation billing

Billing is responsible for calculating the consultation fee according to the patient's age.

The base consultation fee is ₹500.

Patient age	Discount	Expected fee
Below 5 years	100%	₹0
5–17 years	20%	₹400
18–59 years	0%	₹500
60 years and above	40%	₹300

Testing must verify all age boundaries and ensure that the correct fee is displayed and stored.

For example, a patient aged exactly 60 must receive a 40% concession. If the software checks only for an age greater than 60, the patient will be charged incorrectly.

2.4 Pharmacy dispensing

The pharmacy module should allow medicines to be dispensed only when the prescription is valid.

According to the supplied requirement, dispensing is allowed only within seven days of the prescription date.

Testing should include:

- A prescription created on the current date.
- A prescription that is six days old.
- A prescription that is exactly seven days old.
- A prescription that is more than seven days old.
- An expired prescription.
- Incorrect date and time inputs.

The supplied D9 defect demonstrates the importance of date validation. A prescription that is nine days old is accepted because the system incorrectly handles months and days.

2.5 Laboratory reports

The laboratory module should display accurate reports and relevant dates.

Testing should verify:

- Correct patient identification.
- Correct report information.
- Correct collection date.
- Correct report issue date.
- Proper PDF generation and downloading.
- Acceptable page-loading time.

D10 demonstrates that the laboratory PDF displays the collection date instead of the report issue date. D6 highlights a performance problem in which the laboratory report page takes 35 seconds under the specified peak-load scenario.

2.6 Patient reminders

The reminder module sends SMS messages about appointments and changes.

Testing should verify:

- Correct patient details.
- Correct appointment date and time.
- Correct rescheduling information.
- Delivery of messages to the intended recipient.
- Clear and understandable message content.

D7 demonstrates that an unclear rescheduling message can send the old appointment time. This shows that message content must be tested along with the technical delivery process.

3. TESTING AS A CONTINUOUS ENGINEERING ACTIVITY

One of the major conclusions of this assignment is that testing should be performed continuously rather than only at the end of software development.

3.1 Test-last approach

In the test-last approach, developers complete most of the software before the testing team begins detailed testing.

This method may appear simple, but it has several disadvantages:

- Defects may remain undetected for a long time.
- Incorrect requirements may affect multiple modules.
- Fixing defects may become expensive.
- Final testing may take longer than expected.
- Release delays may occur.

For example, if the age concession rule is implemented incorrectly, the defect may affect billing screens, payment calculations and financial reports before it is discovered.

3.2 Review-plus-test approach

This approach combines requirement reviews, design reviews and software testing.

Requirements are checked before implementation. Developers and testers discuss possible misunderstandings. After coding, the software is tested using planned test cases.

This method reduces the possibility of introducing defects and helps identify problems earlier.

3.3 Continuous engineering approach

Continuous engineering includes testing activities throughout development.

Important activities include:

1. Requirement analysis.
2. Requirement reviews.
3. Design verification.
4. Code reviews.
5. Unit testing.
6. Integration testing.
7. System testing.
8. Regression testing.
9. User acceptance testing.
10. Post-release monitoring.

For MediQueue, continuous testing is recommended because the system contains important business rules and interconnected modules.

Conclusion: Continuous testing provides early feedback, reduces the cost of correcting defects and improves confidence in every release.

4. ROLE OF PROCESS IN SOFTWARE QUALITY

A proper software testing process ensures that testing activities are performed in an organised and repeatable manner.

The assignment compared incremental delivery, risk-driven spiral development and DevOps-based automated testing.

4.1 Incremental delivery

In incremental delivery, software is developed and delivered in smaller parts.

For example:

- First increment: Patient registration.
- Second increment: Appointment booking.
- Third increment: Billing.

- Fourth increment: Pharmacy.
- Fifth increment: Laboratory reports.
- Sixth increment: Reminders.

Each increment should be tested before it is integrated into the complete system.

4.2 Risk-driven spiral process

The risk-driven approach gives priority to areas that may cause serious problems.

For MediQueue, high-risk areas include:

- Billing calculations.
- Appointment concurrency.
- Prescription validity.
- Patient information security.
- Payment processing.

This approach helps the team use limited testing time effectively.

4.3 DevOps testing process

DevOps combines development and operations activities. Automated tests can be executed whenever developers integrate new code.

Advantages include:

- Faster feedback.
- Early defect identification.
- Repeatable test execution.
- Improved collaboration.
- Reduced manual regression effort.

4.4 Interpretation of release statistics

The supplied release history contains:

- Total defects: 164.
- Defects found before release: 135.
- Defects reported after release: 29.

Defect Removal Efficiency (DRE):

DRE = Defects found before release / Total defects × 100

DRE = 135 / 164 × 100

DRE = 82.3% approximately.

Escaped defect percentage:

= 29 / 164 × 100

= 17.7% approximately.

These results show that the testing team detected a large proportion of defects before release. However, 29 defects still reached users. The team should investigate why these defects escaped and improve the testing process.

5. IMPORTANCE OF TEST ORACLES AND EXPECTED RESULTS

A test oracle is used to determine the correct expected result of a test case.

Without an accurate expected result, the tester cannot reliably decide whether the software is working correctly.

5.1 Mental calculation

Mental calculation is useful for simple calculations, but it may result in mistakes when many test cases are involved.

5.2 Specification-based oracle

A specification-based oracle uses the approved business requirements to determine the expected result.

For MediQueue, the age concession table provides the expected consultation fee for each age group.

5.3 Independent reference implementation

An independent reference implementation uses a separate program or function to calculate expected results.

It can help generate expected values for multiple test inputs. However, the reference implementation must also be checked to ensure that it follows the approved requirements.

5.4 Importance of D8

D8 demonstrates an incorrect test oracle.

The correct fee for a twelve-year-old patient is ₹400 because the patient receives a 20% concession.

If the test case expects ₹450, the expected result is incorrect.

This example shows that testing itself can contain defects. Therefore, test cases and expected results must be reviewed before they are used as evidence of software quality.

6. APPLICATION OF SOFTWARE TESTING PRINCIPLES

Software testing principles guide testers in selecting suitable test cases and using testing resources effectively.

6.1 Exhaustive testing is impossible

It is generally impossible to test every possible combination of inputs and operations in a large system.

MediQueue has six modules and a total size of 47 KLOC. It serves 9,000 registered patients and has 800 concurrent users during peak usage.

Testing every possible combination would require excessive time and resources.

6.2 Early testing

Testing should begin as early as possible.

Reviewing requirements before coding can identify unclear business rules and reduce the cost of correcting mistakes.

6.3 Defect clustering

Defects may be concentrated in a few modules rather than distributed equally.

The MediQueue modules are:

Module	**Size**
Registration	4 KLOC
Appointment	10 KLOC
Billing	12 KLOC
Pharmacy	9 KLOC
Lab Reports	7 KLOC
Reminders	5 KLOC
Total	**47 KLOC**

Defect density can be calculated as:

Defect density = Number of defects in a module / Module size in KLOC.

This calculation can help identify modules that may require additional investigation. However, defect density should be interpreted together with module complexity, usage and defect severity.

6.4 Pesticide paradox

Repeating the same test cases continuously may eventually stop identifying new defects.

Therefore, test cases should be updated when:

- New requirements are introduced.
- Defects escape into production.
- New risks are identified.
- System architecture changes.
- User feedback reveals new problems.

6.5 Absence-of-errors fallacy

A system may pass many test cases but still fail to satisfy user expectations.

For example, a reminder system may successfully send SMS messages, but the messages may contain incorrect appointment information.

Therefore, passing tests does not automatically mean that the system is suitable for hospital use.

7. IMPORTANCE OF THE TESTER'S ROLE

Testers play an important role in improving software quality.

Their responsibilities include:

- Understanding requirements.
- Preparing test plans.
- Designing test cases.
- Preparing test data.
- Executing tests.

- Recording actual results.
- Reporting defects.
- Retesting corrected defects.
- Performing regression testing.
- Communicating quality risks.

7.1 Independent testing

Independent testers can identify problems that developers may overlook because they are familiar with the implementation.

However, testers must understand the system and its requirements.

7.2 Domain expert involvement

Hospital staff can explain the actual workflow of registration, billing, pharmacy and laboratory activities.

Their involvement helps ensure that the software supports real hospital operations.

7.3 Defect reporting

A good defect report should contain:

1. Defect ID.
2. Defect title.
3. Module name.
4. Environment.
5. Steps to reproduce.
6. Expected result.
7. Actual result.
8. Severity.
9. Priority.
10. Supporting evidence.

A clear defect report improves communication between developers and testers.

7.4 Release responsibility

Testers provide evidence about the quality of the software. The authorised release owner makes the final release decision according to the organisation's process.

Critical defects involving billing, appointment conflicts, patient information or unsafe dispensing should be escalated before release.

8. DEFECT ORIGIN ANALYSIS AND PREVENTION

A defect may originate at different stages of software development.

Common defect origins include:

- Requirement errors.
- Design errors.
- Coding errors.
- Database errors.
- Interface errors.
- Integration errors.
- Test-data errors.
- Deployment errors.

8.1 Requirement-related defects

An unclear requirement may cause developers to implement incorrect behaviour.

For example, the cancellation rule must clearly specify that the ₹100 fee applies only when cancellation occurs less than two hours before the appointment.

8.2 Design-related defects

A design defect may affect the way modules communicate or manage transactions.

D2, involving duplicate billing after a payment-page refresh, requires investigation of transaction handling and duplicate-request protection.

8.3 Coding-related defects

D1 demonstrates an incorrect age condition. D3 demonstrates an incorrect cancellation threshold. D9 demonstrates incorrect prescription date handling.

These examples show why code reviews and boundary-value testing are important.

8.4 Defect prevention

The organisation should focus on preventing defects rather than only correcting them.

Preventive measures include:

- Requirement reviews.
- Design reviews.
- Coding standards.
- Peer code reviews.
- Automated unit tests.
- Integration tests.
- Regression tests.
- Root-cause analysis.
- Developer and tester training.

9. DEFECT CLASSIFICATION AND PRIORITISATION

Defect classification helps the team understand the type and impact of a problem.

The supplied defects can be grouped as follows:

Defect	Main category

D1	Billing calculation
D2	Duplicate transaction
D3	Business-rule logic
D4	Appointment concurrency
D5	Input validation/database handling
D6	Performance
D7	Reminder/message content
D8	Incorrect test oracle
D9	Prescription date validation
D10	Laboratory report display/data mapping

9.1 Severity

Severity indicates the impact of a defect on the system or its users.

A defect that may cause unsafe dispensing or serious appointment conflicts may have high severity.

9.2 Priority

Priority indicates how urgently a defect should be corrected.

A defect may require urgent correction because it affects a frequently used feature, even if its technical severity is moderate.

9.3 Importance of classification

Consistent classification helps:

- Identify high-risk defects.
- Allocate testing resources.
- Plan corrections.
- Monitor recurring problems.
- Improve release decisions.
- Prepare quality reports.

MediQueue should maintain written definitions for severity and priority so that different team members classify defects consistently.

10. IMPORTANCE OF A DEFECT REPOSITORY

A defect repository is a central location where software defects are recorded and managed.

Tools such as Bugzilla and JIRA help teams organise defects and track their progress.

10.1 Main information stored

A defect record should contain:

- Unique defect ID.
- Title and description.
- Module.
- Build version.
- Test environment.
- Reproduction steps.
- Expected result.

- Actual result.
- Severity and priority.
- Assigned developer.
- Current status.
- Date reported.
- Date fixed.
- Retesting result.

10.2 Defect life cycle

A typical defect life cycle is:

New → Triaged → Assigned → In Progress → Fixed → Ready for Retest → Verified → Closed.

If the defect occurs again during retesting, it may be reopened.

10.3 Importance of metrics

Mean Time to Fix helps the organisation understand how long defects take to resolve.

Reopen Rate helps identify defects that were marked fixed but failed during retesting.

These metrics should be interpreted carefully. Fast closure is not useful if the correction has not been properly verified.

11. INTEGRATED TESTING PRACTICE

An integrated testing practice combines different testing activities into one organised strategy.

It includes:

- Requirement reviews.
- Test planning.
- Test design.
- Unit testing.

- Integration testing.
- System testing.
- Performance testing.
- Regression testing.
- Defect management.
- Release evaluation.

MediQueue has a testing budget of 100 person-hours per release and a team consisting of three testers, five developers and one QA lead.

Therefore, testing time should be allocated according to the risk and importance of each module.

11.1 Functional testing

Functional testing verifies whether the system performs the required operations correctly.

Examples include:

- Registering patients.
- Booking appointments.
- Calculating consultation fees.
- Dispensing valid prescriptions.
- Displaying laboratory reports.
- Sending reminders.

11.2 Integration testing

Integration testing checks whether connected modules communicate correctly.

For example, after an appointment is booked, the billing module should receive the correct patient and appointment details.

11.3 Performance testing

Performance testing evaluates the speed and stability of the application.

MediQueue has a response-time target of no more than two seconds. Testing should examine system behaviour under the specified peak usage of 800 concurrent users.

11.4 Regression testing

Regression testing ensures that new changes do not affect previously working features.

For example, changing the billing calculation should not affect appointment booking or payment records.

11.5 User acceptance testing

Hospital staff should verify whether the system supports their actual workflows.

This is important because a system may pass technical tests but still be difficult for staff to use.

12. ADAPTATION TO FUTURE REQUIREMENTS

The expanded MediQueue scenario introduces several new requirements:

- Video teleconsultation.
- Electronic prescriptions.
- National health-ID integration.
- Five hospitals.
- 30,000 patients.
- Weekly releases.
- Audit trail.
- Reduced testing capacity of 50 person-hours.

These changes increase the complexity of the system.

12.1 Video teleconsultation

Testing should include:

- Audio and video quality.
- Internet connectivity.
- Device compatibility.
- Session security.
- Session interruption and recovery.
- Correct patient and doctor connection.

12.2 Electronic prescriptions

Testing should verify:

- Correct prescription creation.
- Authorised access.
- Prescription validity.
- Correct patient identification.
- Accurate medicine information.
- Proper audit records.

12.3 National health-ID integration

Testing should include:

- Identity matching.
- Incorrect or incomplete ID handling.
- External service availability.
- Data privacy.
- Secure communication.
- Recovery after integration failure.

12.4 Audit trail

An audit trail records important activities performed by users and the system.

It should record relevant actions such as:

- Patient record changes.
- Appointment changes.
- Billing transactions.
- Prescription creation and modification.
- Access to sensitive records.

The records should be accurate, appropriately protected and available for authorised review.

12.5 Recommended adaptation

The best approach is to retain the essential testing principles, increase automation and redesign the workflow for weekly releases.

The team should automate stable regression tests and use manual testing for changed requirements, high-risk integrations and exploratory testing.

If a critical feature cannot be tested adequately, it should not be released without appropriate risk approval.

13. GENERAL RECOMMENDATIONS FOR IMPROVING MEDIQUEUE

13.1 Technical recommendations

1. Implement business rules in clearly defined services.
2. Avoid duplicating fee calculation logic across screens.
3. Test all important age boundaries.
4. Test cancellation immediately below, exactly at and above two hours.
5. Use correct rounding logic.
6. Prevent duplicate payment requests.
7. Use suitable database transactions for appointment booking.
8. Validate legitimate patient names and input characters.
9. Test prescription validity around the seven-day boundary.
10. Verify laboratory report dates and labels.
11. Perform realistic performance testing.

12. Protect patient information through appropriate security controls.

13.2 Testing process recommendations

1. Start testing during requirement analysis.
2. Maintain traceability between requirements and test cases.
3. Automate repeatable tests.
4. Perform code reviews.
5. Conduct regular defect triage.
6. Retest every corrected defect.
7. Perform regression testing.
8. Monitor escaped defects.
9. Maintain a central defect repository.
10. Review the testing strategy after every release.

13.3 Team recommendations

The QA lead should coordinate the testing strategy and release evidence.

Developers should perform unit testing and correct identified defects.

Testers should design and execute independent tests and communicate risks.

Hospital domain experts should review workflows and expected behaviour.

The release owner should make decisions based on documented evidence.

14. IMPLEMENTATION ROADMAP

Short-term implementation

During the next release, the team should focus on the known defects and important business rules.

The immediate priorities are:

- Correct the age concession logic.
- Correct the cancellation threshold.
- Prevent duplicate payment transactions.
- Prevent duplicate appointment slots.
- Correct prescription validity checks.
- Resolve laboratory report date errors.
- Investigate the laboratory page performance problem.
- Verify reminder message content.

Every correction should be followed by retesting and appropriate regression testing.

Medium-term implementation

During the next three releases, the team should develop an automated regression test suite for the major patient workflows.

The suite should include registration, appointment booking, billing, pharmacy dispensing, laboratory report access and reminders.

A defect dashboard should be maintained to monitor:

- Defects by module.
- Defects by severity.
- Defect origin.
- Escaped defects.
- Mean time to fix.
- Reopened defects.

Long-term implementation

As the system expands, testing should include:

- Multi-hospital configuration testing.
- External integration testing.

- Security and privacy testing.
- Audit trail verification.
- Backup and recovery testing.
- Performance and scalability testing.
- User acceptance testing.
- Continuous monitoring.

The testing strategy should be reviewed regularly to ensure that it remains suitable for the changing requirements.

15. LIMITATIONS AND ETHICAL CONSIDERATIONS

The assignment provides the main system details, business rules, release statistics and ten named defects. However, it does not provide the complete raw defect history or actual execution records for every testing technique.

Therefore, additional sample defect records, estimated effort values and predicted results should be identified as illustrative rather than real hospital measurements.

The supplied release statistics can be used for calculations, but estimates should not be presented as observed facts.

Testing a hospital system also requires careful handling of patient information. Test data should not unnecessarily expose real patient details. Synthetic or de-identified data should be used where appropriate.

Defects that may affect patient safety, confidentiality or access to healthcare services should be escalated through the appropriate hospital process.

Automation and AI-assisted testing can help generate test cases and identify patterns, but their outputs should be reviewed by qualified team members. An automated test is useful only when its expected result is correct.

16. FINAL CONCLUSION

In conclusion, this assignment demonstrates that software testing is a continuous, systematic and risk-based engineering activity. The quality of MediQueue depends not only on whether individual functions work correctly but also on whether the complete system is reliable, secure, responsive and suitable for hospital use.

The ten tasks show that early requirement reviews, accurate test oracles, boundary-value analysis, risk-based prioritisation, independent testing, defect-origin analysis, consistent defect classification and a central defect repository are essential parts of software quality assurance.

The assignment also highlights the importance of combining manual and automated testing. Automated tests provide fast and repeatable feedback, while manual exploratory testing and domain-expert involvement help identify problems that automated tests may not detect. Functional correctness must be supported by performance, security, privacy, usability, concurrency and integration testing.

For the current MediQueue system, a continuous and risk-driven testing process supported by automated regression testing and formal release gates is recommended. For the expanded system, the same foundation should be strengthened through shift-left testing, improved automation, integration testing, audit verification and careful allocation of the reduced testing budget.

Quality is not the responsibility of only the testing team. Developers, testers, QA leaders, hospital staff and release owners must work together to achieve a dependable product. Testing cannot guarantee that software will never fail, but a disciplined process can reduce the likelihood of failures, detect defects earlier, limit their impact and provide reliable evidence for release decisions.

A successful hospital information system is not defined only by the number of features it provides. It is defined by how accurately, safely and efficiently those features support patients and hospital staff. Therefore, software testing must remain an essential activity throughout the development, deployment and maintenance of MediQueue.

The ultimate goal is to develop a reliable, efficient, secure and user-friendly hospital management system that improves hospital operations and provides a better experience for patients and healthcare professionals.

The MediQueue case demonstrates that software quality depends on the entire development process. Test-last activity is too late for complex billing, appointment and pharmacy rules. Requirement review, continuous testing, risk-based selection, reliable test oracles, independent testing and defect analysis help prevent defects and reduce the cost of repair.

The existing release detected 135 of 164 defects before release, giving a DRE of approximately 82.3%, while 29 defects escaped. The repair-cost data shows why early detection matters: a defect found after release is assumed to cost ₹45,000, compared with ₹800 at requirements/design. The hospital should therefore adopt continuous engineering practices supported by a DevOps pipeline and risk-driven gates.

The recommended operating model is an assured, risk-based practice for patient-safety and revenue-critical modules, with structured testing across the remaining modules. The changed requirements and reduced staffing require a shift-left redesign supported by automation, developer ownership, clinical acceptance and audit-ready evidence. No testing strategy can prove the complete absence of defects; the aim is to reduce risk to an explicitly accepted level.

SOFTWARE TESTING QUALITY CHECKLIST

A testing checklist is used to ensure that important testing activities are completed before releasing software.

S. No.	Testing Activity	Purpose
1	Requirement review	To identify unclear requirements
2	Test planning	To define testing objectives
3	Test case preparation	To cover important scenarios
4	Test data preparation	To provide suitable input values

5	Unit testing	To test individual components
6	Integration testing	To verify communication between modules
7	System testing	To test the complete application
8	Regression testing	To ensure that changes do not break existing functions
9	Performance testing	To measure application speed
10	Security testing	To identify security weaknesses
11	Defect reporting	To record identified problems
12	Defect retesting	To verify corrections
13	User acceptance testing	To check whether the system meets user needs
14	Release approval	To decide whether the application is ready
15	Test documentation	To maintain testing records

Importance of the Checklist

1. Prevents important testing activities from being missed.
2. Improves testing consistency.
3. Helps testers follow a systematic approach.
4. Supports communication between developers and testers.
5. Improves software quality.
6. Provides evidence of completed testing activities.

MediQueue Release Readiness

Before releasing MediQueue, the testing team should confirm that billing, registration, appointments, pharmacy and laboratory report functions have been tested. Critical defects should

be corrected and retested. Performance should also be checked against the required response time of two seconds or less under the specified workload.

A checklist supports release decisions, but it does not guarantee that the software is completely free from defects.

RISK ANALYSIS IN MEDIQUEUE

Risk analysis is the process of identifying possible problems that may affect software quality, patient safety, data security or application performance.

In a hospital management application, risk-based testing is especially important because software errors may affect patient services and hospital operations.

Identification of Risks

The following risks may occur in MediQueue:

1. Incorrect consultation fee calculation.
2. Duplicate payment generation.
3. Incorrect appointment allocation.
4. Loss of patient information.
5. Delayed laboratory reports.
6. Incorrect prescription validation.
7. Failure of appointment reminders.
8. Unauthorised access to patient records.
9. Slow application response during peak hours.
10. Incorrect cancellation fee calculation.

Risk Assessment Table

Risk	Probability	Impact	Priority
Incorrect billing	Medium	High	High
Duplicate payment	Medium	High	High
Double appointment booking	Medium	High	High
Patient data exposure	Low/Medium	Very High	Critical
Slow laboratory reports	High	Medium	High
Incorrect prescription validation	Medium	High	High
SMS reminder failure	Medium	Medium	Medium
Minor display error	Medium	Low	Low

Risk-Based Testing Approach

Risk-based testing gives more attention to functions that have a greater possibility of failure or a greater impact.

For MediQueue:

- Billing should receive high testing priority because incorrect charges affect patients.
- Appointment booking should be tested carefully to prevent double booking.
- Pharmacy functions should be checked to ensure prescription rules are followed.
- Patient registration should be tested for data accuracy.
- Laboratory reports should be tested for correctness and response time.
- Security testing should protect confidential patient information.

Risk Mitigation

Risk mitigation means taking preventive steps to reduce the possibility or impact of problems.

Examples include:

- Reviewing business rules before development.
- Performing code reviews.
- Using automated regression tests.
- Validating all user inputs.
- Testing concurrent appointment requests.
- Maintaining regular database backups.
- Applying access control.
- Monitoring application performance.

TYPES OF SOFTWARE TESTING APPLICABLE TO MEDIQUEUE

Different testing techniques are required to evaluate the functionality, performance, security and reliability of MediQueue.

Unit Testing

Unit testing checks individual functions or components separately.

Example: Testing the consultation fee calculation function for different patient age groups.

Integration Testing

Integration testing checks whether two or more modules communicate correctly.

Example: Checking whether the appointment module correctly sends appointment details to the reminder module.

System Testing

System testing evaluates the complete application as a whole.

Example: Registering a patient, booking an appointment, generating a bill and accessing the laboratory report.

Functional Testing

Functional testing checks whether the application performs according to the specified requirements.

Example: Verifying that patients aged 60 years and above receive a 40% consultation fee concession.

Regression Testing

Regression testing checks whether new modifications have affected previously working functions.

Example: After modifying the billing module, previously working appointment and payment functions should also be checked.

Performance Testing

Performance testing evaluates the speed, stability and responsiveness of the application.

Example: Checking whether MediQueue meets its response-time requirement of two seconds or less under the specified workload.

Load Testing

Load testing checks application behaviour under an expected number of users.

Example: Testing the application with 800 concurrent users.

Stress Testing

Stress testing evaluates application behaviour beyond normal operating conditions.

Example: Increasing the number of simultaneous users beyond the expected peak to identify system limitations.

Security Testing

Security testing identifies weaknesses that may allow unauthorised access or data exposure.

Example: Checking whether a patient can access another patient's medical records.

Usability Testing

Usability testing checks whether the application is easy to understand and operate.

Example: Checking whether patients can easily register, book appointments and view reports.

Compatibility Testing

Compatibility testing checks whether the application works across different browsers, devices and operating systems.

Example: Testing MediQueue using different browsers and mobile screen sizes.

Acceptance Testing

Acceptance testing determines whether the application meets the needs of its intended users.

Example: Hospital staff and selected users verify whether the registration, billing and appointment workflows meet their requirements.

Summary

Using multiple testing types provides a broader evaluation of MediQueue. Functional testing checks correctness, performance testing checks speed, security testing protects information and usability testing improves the user experience.

ETHICAL AND PROFESSIONAL RESPONSIBILITIES

Software testing in healthcare applications requires special care because the system handles confidential patient information and supports important hospital activities.

Patient Privacy

Testers must protect patient information and should avoid using real patient data unnecessarily. Synthetic or properly anonymised data should be used whenever possible.

Data Confidentiality

Patient records, laboratory reports, prescriptions and payment details should be accessible only to authorised users.

Accuracy of Information

Testers should verify that patient details, billing amounts, appointment dates and laboratory report information are displayed correctly.

Responsible Defect Reporting

Every identified defect should be reported clearly. Testers should not hide serious defects to meet release deadlines.

Professional Communication

Testers should communicate defects respectfully and provide enough information for developers to reproduce and correct the problem.

Patient Safety

Errors involving billing, appointments, prescriptions or medical reports should receive appropriate attention because they may affect patient services.

Honest Documentation

Test results must represent actual testing activities. Illustrative results, expected results and executed results should be clearly distinguished.

Role of Test Automation in Future Development

Test automation can improve the efficiency of the MediQueue testing process.

Automated scripts can be prepared for:

1. Patient registration.
2. Consultation fee calculation.
3. Appointment booking.
4. Appointment cancellation.
5. Prescription validation.
6. Laboratory report access.
7. User authentication.
8. Regression testing.

Automation does not eliminate the need for manual testing. Human testers are still required to understand requirements, identify unusual situations and evaluate usability.

Importance of Continuous Improvement

Continuous improvement is the process of regularly identifying weaknesses and making changes to improve software quality.

The MediQueue team can improve its testing process by:

- Reviewing defects after every release.
- Identifying frequently affected modules.
- Improving test case coverage.
- Automating repeated test cases.
- Conducting regular code reviews.
- Updating testing documentation.

- Collecting feedback from users.
- Monitoring application performance.

Continuous improvement helps the organisation learn from previous mistakes and prevent similar defects in future releases.

Importance of User Feedback

User feedback is an important source of information for improving software applications.

Patients may report difficulties during registration, appointment booking or payment. Doctors and hospital staff may identify problems related to patient records, prescriptions or laboratory reports.

The development team should collect feedback, analyse the reported problems and make suitable improvements.

User feedback helps ensure that the application meets real-world requirements.

Overall Learning Outcomes

Through this assignment, the following learning outcomes can be achieved:

1. Understanding the basic concepts of software testing.
2. Learning the importance of software quality.
3. Understanding different software development and testing processes.
4. Preparing test cases based on business requirements.
5. Identifying errors, defects and failures.
6. Understanding defect classification and prioritisation.
7. Learning how to maintain a defect repository.
8. Understanding the importance of risk-based testing.
9. Comparing manual and automation testing.
10. Applying software testing concepts to a real-world healthcare application.

KEY FORMULAS

Metric	Formula
Fault detection percentage	Faults detected ÷ faults injected × 100
Faults found per person-hour	Faults detected ÷ testing effort in person-hours
Escaped-fault cost	Number of escaped faults × cost per post-release repair
Defect Removal Efficiency	Defects found before release ÷ total defects × 100
Defect escape rate	Defects reported after release ÷ total defects × 100
Defects per KLOC	Number of defects in module ÷ module size in KLOC
Mean time to fix	Total elapsed repair time ÷ number of verified closed defects
Reopen rate	Reopened defects ÷ defects marked fixed × 100
Oracle error rate	Incorrect expected values from oracle ÷ total oracle outputs checked × 100
Classification agreement	Matching classifications ÷ total defects classified × 100

REFERENCES

1. Software Testing Assignment 1 – CO1, Lakeside Hospital MediQueue scenario and task descriptions, faculty-provided document.
2. Myers, G. J., Sandler, C., & Badgett, T. The Art of Software Testing. Wiley.
3. ISTQB. Certified Tester Foundation Level Syllabus. International Software Testing Qualifications Board.
4. ISO/IEC/IEEE 29119. Software Testing – Concepts, Processes and Documentation.
5. ISO/IEC 25010. Systems and Software Quality Models.

6. OWASP. Application Security Verification Standard (ASVS) and Web Security Testing guidance.
7. Python Software Foundation. Python 3 documentation: unittest and decimal modules.
8. pytest documentation. Fixtures, parametrization and assertion reporting.
9. Atlassian documentation. Jira issue types, workflows and dashboards.
10. Bugzilla documentation. Defect tracking, components, statuses and queries.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
QA	Quality Assurance
QC	Quality Control
SDLC	Software Development Life Cycle
STLC	Software Testing Life Cycle
DRE	Defect Removal Efficiency
UI	User Interface
DB	Database
API	Application Programming Interface
SQL	Structured Query Language
KLOC	Thousand Lines of Code
UAT	User Acceptance Testing
CI	Continuous Integration
CD	Continuous Delivery / Deployment
GUI	Graphical User Interface
MTTR	Mean Time to Repair
P0	Critical Priority

P1	High Priority
P2	Medium Priority
P3	Low Priority
LLM	Large Language Model
EHR	Electronic Health Record
PDF	Portable Document Format

APPENDIX

11. The appendix contains supporting information, sample test cases, calculations, defect records and other materials used in this assignment.

12. Appendix A – MediQueue System Details

- Application name: MediQueue
- Frontend: Angular
- Backend: Node.js
- Database: PostgreSQL
- Total application size: 47 KLOC
- Total modules: Six
- Registered patients: 9,000
- Doctors and staff: 250
- Peak concurrent users: 800
- Expected response time: 2 seconds or less

Appendix B – Important Business Rules

1. Standard consultation fee: ₹500.
2. Children below 5 years: Free consultation.
3. Patients aged 5–17 years: 20% concession.
4. Patients aged 18–59 years: No concession.
5. Patients aged 60 years and above: 40% concession.
6. Final fee is rounded to the nearest rupee, with 0.5 rounded upward.
7. Cancellation less than 2 hours before the appointment attracts a ₹100 fee.

8. A patient can have a maximum of two active appointments.
9. Prescriptions can be dispensed only within seven days.

Appendix C – Sample Test Case Format

13. Test Case ID	14. Test Scenario	15. Expected Result	16. Actual Result	17. Status
18. TC01	19. Patient below 5 years	20. Free consultation	21. To be entered after execution	22. Pending
23. TC02	24. Patient aged 12 years	25. ₹400 consultation fee	26. To be entered after execution	27. Pending
28. TC03	29. Patient aged 60 years	30. ₹300 consultation fee	31. To be entered after execution	32. Pending
33. TC04	34. Cancellation 3 hours before appointment	35. No late cancellation fee	36. To be entered after execution	37. Pending
38. TC05	39. Cancellation 1 hour before appointment	40. ₹100 late cancellation fee	41. To be entered after execution	42. Pending

Appendix D – Sample Defect Report

- Defect ID: D1
- Module: Billing
- Defect title: Incorrect consultation fee for a patient aged 60 years.
- Expected result: ₹300.
- Actual result: ₹500.
- Severity: High.

- Priority: High.
- Status: Open until corrected and retested.

Appendix E – Supporting Evidence

Attach the following evidence if required by your faculty:

1. Screenshots of test execution.
2. Python program output.
3. Sample test case execution results.
4. Defect tracking screenshots.
5. Fault-tree diagrams.
6. Testing process flowcharts.
7. Tables showing defect analysis and calculations.